

Concorrência e sincronização

Condição de corrida, semáforos, produtor-consumidor e monitores

Weslei Ferreira Santos

7 de outubro de 2026

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Parte 1: teoria

1. De processos a threads
2. Condição de corrida
3. Região crítica e Peterson
4. Semáforos
5. Produtor-consumidor em C
6. Monitores em Java

Parte 2: demonstração visual

7. Simulação “Produção de dinheiro”

Prática e fechamento

8. Seis lacunas no produtor-consumidor
9. Bilhete de saída

Se as threads compartilham a memória, como impedir que uma atrapalhe a outra?

Ao final, você deve conseguir:

- explicar por que `contador++` em duas threads perde incrementos;
- definir região crítica e exclusão mútua;
- dizer o que `mutex`, `empty` e `full` controlam, e por que a ordem importa;
- explicar `synchronized`, `wait` e `notify`, e por que usar `while`;
- completar o produtor-consumidor com os semáforos certos.

De processos a threads

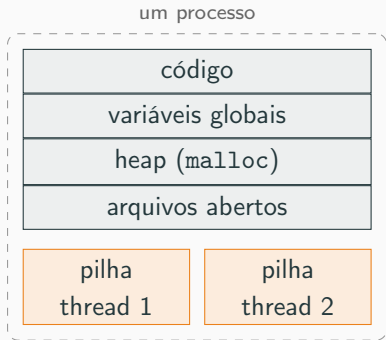
Lembrando a aula passada

- No `fork2.c`, o pai somou 0.
- Cada processo tem a **sua** memória: o que um escreve, o outro não vê.
- Para conversar, foi preciso um canal do SO: o **pipe**.

Hoje o problema se inverte

As **threads** de um processo **dividem** a memória. Comunicar ficou fácil. O difícil agora é **não se atrapalhar**.

O que as threads compartilham



Compartilhado (azul): código, globais, heap, arquivos.

De cada thread (laranja): pilha (variáveis locais) e registradores.

Consequência

Uma variável global escrita por uma thread é vista pela outra na hora, sem pipe.

Criando threads em C

```
void *somar(void *arg) { ... return NULL; }    /* o que a thread executa */

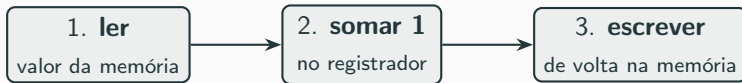
pthread_t t1, t2;
pthread_create(&t1, NULL, somar, NULL);        /* cria e ja comeca a rodar */
pthread_create(&t2, NULL, somar, NULL);
pthread_join(t1, NULL);                       /* espera a thread terminar */
pthread_join(t2, NULL);
```

	Processos (aula passada)	Threads (hoje)
Criar	fork()	pthread_create()
Esperar terminar	waitpid()	pthread_join()
Memória	cópia separada	a mesma
Compilar	normal	com -pthread

Condição de corrida

contador++ não é uma operação só

Para o processador, contador++ são **três passos**:



O perigo

Entre um passo e outro, o escalonador pode trocar de thread. A outra thread pode ler o valor **antigo** antes da primeira escrever o novo.

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
-------	----------	----------	----------

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0
3	calcula 1		0

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0
3	calcula 1		0
4		calcula 1	0

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0
3	calcula 1		0
4		calcula 1	0
5	escreve 1		1

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0
3	calcula 1		0
4		calcula 1	0
5	escreve 1		1
6		escreve 1	1

Uma intercalação que perde um incremento

Contador começa em 0. Cada thread quer somar 1. Esperado: 2.

Passo	Thread A	Thread B	contador
1	lê 0		0
2		lê 0	0
3	calcula 1		0
4		calcula 1	0
5	escreve 1		1
6		escreve 1	1

Condição de corrida

O resultado depende da **ordem** em que as threads foram intercaladas. Aqui, a escrita de A foi apagada pela de B.

Na prática: corrida.c

```
static volatile int contador = 0;           /* global: as duas threads veem a mesma */

static void *somar(void *arg) {
    for (int i = 0; i < 1000000; ++i)
        contador = contador + 1;           /* regiao critica SEM protecao */
    return NULL;
}
```

```
$ gcc -Wall -Wextra examples/dia07/corrida.c -o corrida -pthread
$ ./corrida
Contador: 1006933 (esperado: 2000000)
$ ./corrida
Contador: 1349235 (esperado: 2000000)
```

Quase metade dos incrementos se perde, e o resultado **muda a cada execução**.

Dois avisos sobre `corrida.c`

- Em C, duas threads acessando a mesma variável sem sincronização é uma **data race**: comportamento **indefinido**. O resultado não está garantido nem a ser “1 ou 2”.
- `volatile` **não conserta**: ele só impede o compilador de juntar as somas. Está ali para a corrida aparecer.
- Pôr um `sleep` também não conserta: só muda a probabilidade.

O que conserta

Garantir que só uma thread por vez execute o trecho que mexe no contador. Isso tem nome: **exclusão mútua**.

Região crítica e exclusão mútua

Região crítica e exclusão mútua

Definições

Região crítica: trecho do código que acessa um recurso compartilhado.

Exclusão mútua: no máximo **uma** thread por vez dentro da região crítica.

Uma boa solução precisa garantir (Tanenbaum):

1. Duas threads nunca estão ao mesmo tempo na região crítica.
2. Não depende da velocidade nem do número de CPUs.
3. Quem está fora da região crítica não bloqueia ninguém.
4. Ninguém espera para sempre para entrar.

Algoritmo de Peterson (duas threads)

```
/* entrada */
interessado[eu] = verdadeiro
vez = outro
enquanto interessado[outro] e vez == outro:
    esperar

/* região crítica */

/* saída */
interessado[eu] = falso
```

- interessado[i]: a thread i quer entrar.
- vez: quem cede em caso de empate.

Situação	O que acontece
O outro não quer entrar	Entro direto
Os dois querem	Quem escreveu vez por último espera

Peterson em C: peterson-code.c

```
static void enter_region(int eu) {  
    int outro = 1 - eu;  
    atomic_store(&interested[eu], 1);           /* quero entrar */  
    atomic_store(&turn, outro);                  /* cedo a vez em caso de empate */  
    while (atomic_load(&interested[outro]) && atomic_load(&turn) == outro)  
        ;                                         /* espera ocupada */  
}  
static void leave_region(int eu) { atomic_store(&interested[eu], 0); }
```

```
$ gcc -Wall -Wextra peterson-code.c -o peterson -pthread && ./peterson  
Contador: 20000 (esperado: 20000)
```

Por que atomics? Com variáveis comuns, compilador e processador podem reordenar leituras e escritas, e Peterson deixa de funcionar.

O que a thread faz enquanto está presa no `while`?

O que a thread faz enquanto está presa no `while`?

- Fica **girando**: testa, testa, testa de novo.
- Continua **gastando CPU** sem fazer trabalho útil.
- Com uma CPU só, pode até atrasar a thread que precisa sair da região crítica.

Queremos melhor

Em vez de girar, a thread deveria **dormir** e ser **acordada** quando puder seguir. É isso que o semáforo faz.

Semáforos

Semáforo: um contador que nunca fica negativo

`sem_wait(&s)` (também: *down*, P)

- Se $s > 0$: diminui 1 e segue.
- Se $s == 0$: a thread **dorme**.

`sem_post(&s)` (também: *up*, V)

- Soma 1.
- Se alguém dorme em s , **acorda** um.

Analogia: estacionamento com placa de vagas

O valor do semáforo é o número de vagas na placa. Quem chega e vê 0 **espera parado**, sem rodar em volta do quarteirão. Quem sai, soma uma vaga e libera o próximo.

A espera e o despertar são feitos pelo SO de forma **atômica**: não há corrida no próprio contador.

Os três semáforos do produtor-consumidor

Semáforo	Começa em	O que controla
mutex	1	Exclusão mútua: só uma thread mexe no buffer por vez. É a “chave”.
empty	N	Disponibilidade: quantas vagas livres existem.
full	0	Disponibilidade: quantos itens prontos existem.

Dois problemas diferentes

mutex responde “posso mexer agora?”.

empty e full respondem “tem o que eu preciso?”.

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consome A	2	1	1

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consume A	2	1	1
produz C	1	2	2

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consume A	2	1	1
produz C	1	2	2
produz D	0	3	3

Simulando no quadro com $N = 3$

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consume A	2	1	1
produz C	1	2	2
produz D	0	3	3

E se o produtor quiser o quinto item? Ele dorme em `sem_wait(&empty)`, sem segurar o mutex, até o consumidor liberar uma vaga.

Simulando no quadro com $N = 3$

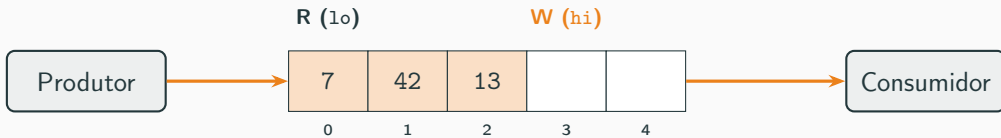
Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consome A	2	1	1
produz C	1	2	2
produz D	0	3	3

E se o produtor quiser o quinto item? Ele dorme em `sem_wait(&empty)`, sem segurar o mutex, até o consumidor liberar uma vaga.

Repare: `empty + full = N` sempre que nenhuma operação está em andamento.

Produtor-consumidor em C

O problema: um buffer, dois ritmos



- O produtor escreve na posição hi ; o consumidor lê da posição lo .
- Os índices andam com $(i + 1) \% N$: depois da última posição, voltam para a 0 (**buffer circular**).
- Buffer **cheio**: o produtor precisa esperar. Buffer **vazio**: o consumidor precisa esperar.
- Ninguém pode mexer no buffer enquanto o outro mexe.

Produtor e consumidor: prod_cons.c

```
/* PRODUTOR */  
wait_sem(&empty); /* 1. reserva VAGA */  
wait_sem(&mutex); /* 2. pega a chave */  
insert_item(item); /* 3. mexe no buffer */  
sem_post(&mutex); /* 4. devolve chave */  
sem_post(&full); /* 5. avisa: ITEM */
```

```
/* CONSUMIDOR */  
wait_sem(&full); /* 1. reserva ITEM */  
wait_sem(&mutex); /* 2. pega a chave */  
item = remove_item(); /* 3. mexe */  
sem_post(&mutex); /* 4. devolve chave */  
sem_post(&empty); /* 5. avisa: VAGA */
```

Regra para lembrar

Cada thread **espera o que consome** e **avisa o que cria**. O produtor consome vagas e cria itens; o consumidor, o contrário.

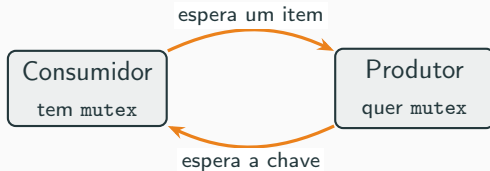
```
$ gcc -Wall -Wextra prod_cons.c -o prod_cons -pthread && ./prod_cons  
Produzido: 29 | ocupacao: 1/20 ... Fim: 40 itens produzidos e consumidos; buffer vazio.
```

E se a ordem for invertida?

```
/* CONSUMIDOR ERRADO */  
wait_sem(&mutex);  /* pega a chave...  
    */  
wait_sem(&full);    /* ...e dorme com  
    ela */
```

Com o buffer **vazio**:

1. O consumidor pega o mutex.
2. Dorme em full, **segurando a chave**.
3. O produtor dorme em mutex.



Deadlock

Cada um espera algo que só o outro pode dar. O programa trava para sempre.

Monitores em Java

Por que monitores?

Semáforos funcionam, mas deixam tudo nas mãos do programador:

- É fácil **esquecer** um `sem_post`: alguém dorme para sempre.
- É fácil **trocar a ordem**: deadlock, como acabamos de ver.
- É fácil **proteger a coisa errada**: a corrida continua.

Monitor

Um objeto que junta os **dados compartilhados**, as **operações** sobre eles e a **exclusão mútua**. Quem usa o objeto não precisa lembrar de pegar e devolver a chave.

Analogia: um banheiro com uma **chave única**. Quem tem a chave entra; os outros esperam na porta.

As três ferramentas de um monitor Java

Recurso	O que faz
<code>synchronized</code>	Para executar o método, a thread precisa da chave (lock) do objeto. Só uma thread por vez dentro dos métodos <code>synchronized</code> do mesmo objeto.
<code>wait()</code>	Solta a chave e dorme até ser avisada. Só pode ser chamado com a chave na mão.
<code>notify()</code>	Acorda uma thread que esperava nesse objeto. Ela ainda precisa pegar a chave de novo para continuar.
<code>notifyAll()</code>	Acorda todas ; cada uma confere a sua condição.

`Thread.sleep()` **não** solta a chave. Só `wait()` solta.

Mailbox 3b: sem sincronização

```
public void storeMessage(String msg) {           // dois produtores chamam ao mesmo tempo
    for (int i = 0; i < msg.length(); i++) {
        message[i] = msg.charAt(i);              // escreve letra por letra no MESMO array
        Thread.sleep((int) (MAXPROCESSTIME * Math.random()));
    }
    youHaveMail = true;
}
```

Saída real (dois produtores: Dave diz “Hello, world.” e Bill diz “Hot dog!”):

```
My name is, Bill. I say, Hello, worl
My name is, Bill. I say, Hot dog!    .
My name is, Dave. I say, Het dog!
```

As letras dos dois produtores se misturam no mesmo array: é a condição de corrida, agora em Java.

Mailbox 3s: com monitor

```
public synchronized void storeMessage(String msg) {    // precisa da chave do objeto
    ... copia a mensagem letra por letra ...
    youHaveMail = true;
    notify();                                           // acorda o consumidor
}

public synchronized String retrieveMessage() {
    while (youHaveMail == false)
        wait();                                       // solta a chave e dorme
    String msg = new String(message, 0, MAXMESSAGE);
    youHaveMail = false;
    return msg;
}
```

Agora um produtor só começa a escrever quando o outro terminou: as mensagens saem **íntegras**. O consumidor dorme em vez de perguntar repetidamente.

Por que `while` e não `if` antes de `wait()`?

- Quem acorda **não recebe a chave na hora**: precisa disputá-la de novo.
- Enquanto isso, outra thread pode entrar e **mudar a condição** (por exemplo, pegar a mensagem antes).
- O Java também permite **despertares espúrios**: acordar sem ninguém ter chamado `notify`.

Regra

Ao acordar, **confira de novo**. Com `while`, a thread só segue quando a condição for verdadeira de fato.

O limite da mailbox 3s

Ela ainda pode perder mensagens

storeMessage não espera a caixa esvaziar. Se dois produtores escrevem antes de o consumidor ler, a primeira mensagem é **sobrescrita**. A 3s garante **integridade**, não entrega.

Como seria a versão completa (uma posição, sem perda):

<code>guardar (synchronized):</code>	<code>retirar (synchronized):</code>
<code> enquanto cheia: wait()</code>	<code> enquanto vazia: wait()</code>
<code> copia a mensagem</code>	<code> copia a mensagem</code>
<code> cheia = verdadeiro</code>	<code> cheia = falso</code>
<code> notifyAll()</code>	<code> notifyAll()</code>

É o produtor-consumidor com $N = 1$, escrito com monitor em vez de semáforos.

Parte 2: demonstração visual

Simulação “Produção de dinheiro”

Na tela	No <code>prod_cons.c</code>
10 posições	N
W: próxima escrita	hi
R: próxima leitura	lo
itens disponíveis	full
vagas livres	empty
“Aguardando. . .”	dormindo num semáforo

- **Banco:** o produtor (cria notas).
- **Carro-forte:** o consumidor (retira notas).
- Velocidades de 0,2 a 10 itens/s.
- **Iniciar, Pausar e Reiniciar.**

Combinado

Antes de cada mudança, **faça a sua previsão.**

Tudo o que aparecer na tela tem um semáforo do C por trás.

Experimento 1: ritmos iguais

Produtor 2 itens/s · Consumidor 2 itens/s

Previsão: o buffer vai encher?

Experimento 1: ritmos iguais

Produtor 2 itens/s · Consumidor 2 itens/s

Previsão: o buffer vai encher?

- Não: fica oscilando com poucas notas.
- W e R andam quase juntos.
- Ninguém espera muito: há quase sempre vaga e quase sempre item.

Experimento 2: produtor rápido

Produtor 10 itens/s · Consumidor 0,2 itens/s

Previsão: o que acontece? Quem vai esperar?

Experimento 2: produtor rápido

Produtor 10 itens/s · Consumidor 0,2 itens/s

Previsão: o que acontece? Quem vai esperar?

- O buffer enche e aparece “**Produtor: Aguardando vaga: buffer cheio**”.

Experimento 2: produtor rápido

Produtor 10 itens/s · Consumidor 0,2 itens/s

Previsão: o que acontece? Quem vai esperar?

- O buffer enche e aparece “**Produtor: Aguardando vaga: buffer cheio**”.

No código C

O produtor está dormindo em `wait_sem(&empty)`, com `empty = 0`. Ele **não** segura o mutex: por isso o consumidor continua conseguindo retirar.

Experimento 3: consumidor rápido

Produtor 0,2 itens/s · Consumidor 10 itens/s

Previsão: e agora?

Experimento 3: consumidor rápido

Produtor 0,2 itens/s · Consumidor 10 itens/s

Previsão: e agora?

- O buffer esvazia e aparece “Consumidor: Aguardando item: buffer vazio”.

Experimento 3: consumidor rápido

Produtor 0,2 itens/s · Consumidor 10 itens/s

Previsão: e agora?

- O buffer esvazia e aparece “Consumidor: Aguardando item: buffer vazio”.

No código C

O consumidor está dormindo em `wait_sem(&full)`, com `full = 0`. Cada nota nova faz `sem_post(&full)` e acorda o consumidor.

Experimento 4: a volta do buffer e a pausa

Produtor 3 · Consumidor 3, depois Pausar

Previsão: o que acontece com W depois da posição 9? E quanto dá itens + vagas?

Experimento 4: a volta do buffer e a pausa

Produtor 3 · Consumidor 3, depois Pausar

Previsão: o que acontece com W depois da posição 9? E quanto dá itens + vagas?

- W volta para a posição 0: é o $h_i = (h_i + 1) \% N$.
- Com a simulação pausada: **itens + vagas = 10**, sempre.
- É o `empty + full = N` da simulação no quadro.

Experimento 4: a volta do buffer e a pausa

Produtor 3 · Consumidor 3, depois Pausar

Previsão: o que acontece com W depois da posição 9? E quanto dá itens + vagas?

- W volta para a posição 0: é o $hi = (hi + 1) \% N$.
- Com a simulação pausada: **itens + vagas = 10**, sempre.
- É o `empty + full = N` da simulação no quadro.

Para fechar a parte 2

O mutex garante que ninguém atrapalha. `empty` e `full` garantem que ninguém trabalha sem ter o que precisa.

Prática

Seis lacunas: prod_cons_atividade.c

```
sem_init(&mutex, 0, 1);  
sem_init(&empty, 0, /* 1: vagas no inicio */);  
sem_init(&full, 0, /* 2: itens no inicio */);
```

```
/* produtor */  
wait_sem(/* 3 */);  
wait_sem(&mutex);  
insert_item(item);  
sem_post(&mutex);  
sem_post(/* 4 */);
```

```
/* consumidor */  
wait_sem(/* 5 */);  
wait_sem(&mutex);  
item = remove_item();  
sem_post(&mutex);  
sem_post(/* 6 */);
```

1 e 2: quantas vagas e quantos itens no início?

3 e 5: o que cada um precisa antes de mexer?

4 e 6: o que cada um avisa depois?

O mutex já está pronto. Não mude a ordem das linhas.

Compilar, testar e diagnosticar

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    examples/dia07/prod_cons_atividade.c -o prod-cons-aluno -pthread
$ ./prod-cons-aluno
...
Fim: 40 itens produzidos e consumidos; buffer vazio.
```

Sintoma

O que significa

Trava sem terminar (Ctrl + C)

Alguém espera um semáforo que ninguém libera

Assertion 'count < N' failed

O produtor inseriu sem haver vaga

Assertion 'count > 0' failed

O consumidor retirou sem haver item

Antes de preencher, o esqueleto **não compila**. É proposital.

Fechamento

Se você pensou...	Reveja...
"contador++ é uma operação só."	São três passos: ler, somar, escrever.
"volatile resolve a corrida."	Não: é preciso exclusão mútua.
"O mutex conta as vagas."	O mutex é a chave; quem conta é empty e full.
"Tanto faz a ordem dos sem_wait."	Dormir segurando a chave causa deadlock.
"Semáforo é igual a Peterson."	No semáforo quem espera dorme; em Peterson, gira.
"notify entrega a chave na hora."	Quem acorda disputa a chave e confere de novo.
"sleep libera o monitor."	Só wait libera.
"A mailbox 3s entrega tudo."	Ela pode sobrescrever mensagem não lida.

Condição de corrida	Resultado depende da ordem de intercalação das threads.
Região crítica	Trecho que acessa o recurso compartilhado.
Peterson	Exclusão mútua para duas threads, com espera ocupada.
Semáforo	Contador com wait/post; quem espera dorme.
Produtor-consumidor	mutex para exclusão; empty e full para disponibilidade.
Monitor	Dados, operações e exclusão mútua juntos; wait/notify para condições.

As duas aulas em uma frase

Processos isolam a memória e precisam de pipe para conversar. Threads compartilham e precisam de sincronização para não se atrapalhar.

Individual, em duas ou três frases:

Qual problema o mutex resolve
e qual problema empty e full resolvem?

Thread: fluxo de execução dentro de um processo; divide a memória com as outras.

Condição de corrida: resultado depende da ordem das threads.

Data race: acesso concorrente sem sincronização; em C, comportamento indefinido.

Região crítica: trecho que mexe no recurso compartilhado.

Exclusão mútua: uma thread por vez na região crítica.

Espera ocupada: esperar girando e gastando CPU.

Semáforo: contador não negativo com wait e post.

Mutex: semáforo (ou lock) que vale 0 ou 1; a “chave”.

Deadlock: cada um espera algo que só o outro pode dar.

Monitor: objeto com dados, operações e exclusão mútua juntos.

wait / notify: dormir soltando a chave / acordar quem dorme.

Buffer circular: índices voltam a 0 com % N.

Cola: compilar e executar os exemplos

Na raiz do repositório:

```
$ gcc -Wall -Wextra examples/dia07/corrida.c -o corrida -pthread && ./corrida
$ gcc -Wall -Wextra peterson-code.c -o peterson -pthread && ./peterson
$ gcc -Wall -Wextra prod_cons.c -o prod_cons -pthread && ./prod_cons
$ make java && timeout 5 java -cp build/java-3b ThreadSync3b      # mailbox 3b
$ make java && timeout 5 java -cp build/java-3s ThreadSync3s      # mailbox 3s
$ make run-simulation                                           # Producao de dinheiro

# pratica (avisos viram erros):
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    examples/dia07/prod_cons_atividade.c -o prod-cons-aluno -pthread && ./prod-cons-aluno
```

As mailboxes Java nunca terminam sozinhas: o timeout 5 encerra depois de 5 segundos (ou use Ctrl + C).

Rode os exemplos: os comandos estão no slide anterior.

Leia

- A **apostila** da aula: os mesmos temas, com mais calma, e exercícios com respostas.
- `man 3 sem_wait`, `man 3 pthread_create`, `man 7 pthreads`.
- Tanenbaum e Bos, *Sistemas Operacionais Modernos*, 4. ed., seção 2.3 (comunicação entre processos).
- Silberschatz, Galvin e Gagne, *Fundamentos de Sistemas Operacionais*, 9. ed., cap. 5 (sincronização de processos).

Dúvidas?

Weslei Ferreira Santos
MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Material-base: Prof. Maycon Leone M. Peixoto